
OpenBinding4Placement: QoS-aware Placement of FaaS Compositions in the Cloud–Edge Continuum

Technical Report

Version 1.0

Anonymous Author(s)

Anonymous Institution(s)

Supplementary material for a double-blind submission to the
24th International Conference on Service-Oriented Computing (ICSOC 2026)

July 20, 2026

INDEX

1	Introduction	1
2	Background	3
2.1	FaaS composition and placement	3
2.2	The binding problem	3
2.3	SaaS and FaaS pricings	3
3	OpenBinding4Placement: Implementation	5
3.1	Architecture	5
3.2	From FaaS pricing to candidate costs	6
3.3	QACO' and BIM': a placement-aware binding model	9
3.3.1	The BIM' serialization	9
3.4	Solving engines	10
3.5	The reference evaluator	11
4	Experimental Design	12
4.1	Research questions	12
4.2	Source dataset	12
4.3	Corpus generation: from SECFAAS2FOG to priced BIM' instances	12
4.4	Campaign protocol	14
5	Evaluation	18
5.1	Technical validation	18
5.2	RQ ₁ : Feasibility versus optimization	18
5.3	RQ ₂ : Quality, runtime, and scalability	22
5.4	RQ ₃ : The time cost of optimization	24
5.5	RQ ₄ : Suitability for runtime rebinding	27
6	Limitations and Threats to Validity	29
A	Reproducibility Guide	30
A.1	Repository layout	30
A.2	Running the pipeline	30
A.3	Generated artifacts	31

Chapter 1

Introduction

Function-as-a-Service (FaaS) applications are increasingly deployed across the Cloud–Edge continuum, where functions may run on edge devices, fog nodes, or cloud resources. This distribution can reduce latency and exploit heterogeneous resources, but it also makes placement decisions more complex: a valid placement must satisfy hardware and software requirements, latency constraints, service dependencies, security policies, and provider-specific conditions.

Existing approaches such as SECFAAS2FOG [1] address this problem from a feasibility perspective, determining *eligible* placements of secure FaaS compositions over Cloud–Edge infrastructures. However, several placements may be valid while differing significantly in QoS attributes such as cost, latency, or resource consumption. Placement should therefore be treated as an *optimization* problem: among all valid deployments, the system should select the one that best satisfies the objectives. This is particularly relevant when pricing is considered: FaaS costs depend on provider-specific pricing models, execution regions, allocated resources, invocation rates, execution time, and workload conditions, so an initially cost-effective placement may become suboptimal as these evolve. We refer to this optimization problem as CLASP-FaaS (CLASP for brevity): the Cost- and Latency-Aware Secure Placement of FaaS Compositions in the Cloud–Edge Continuum.

CLASP is given an operational semantics by formulating it as a placement-aware extension of the QoS-aware service composition (QACO) problem. Under this formulation, functions are modeled as abstract tasks, execution locations as candidate services, and placement requirements as constraints and quality attributes. The resulting optimization problem is solved by OpenBinding4Placement, an extension of OpenBinding [3], an API gateway for solving QACO problems through interchangeable engines.

This technical report is the laboratory package companion of the paper *QoS-aware Placement of FaaS Compositions in the Cloud-Edge Continuum*. While the paper concentrates on the approach and headline results, this report documents the material that did not fit there:

- a detailed description of the OpenBinding4Placement implementation — the gateway architecture, the FaaS-oriented iPricing models and their experimental variables extension, the QACO'/BIM' placement-aware binding model, the three extended solving engines, and the reference evaluator that makes their results comparable (Chapter 3);
- the full experimental pipeline — how the SECFAAS2FOG dataset is transformed into a priced, certified-satisfiable BIM' corpus of 105 instances, and how the 2 205-run campaign is executed and audited (Chapter 4);
- the complete evaluation — the technical validation of the pipeline and the four research questions of the paper, illustrated with the full set of figures produced by the analysis notebooks, including those omitted from the paper for space reasons (Chapter 5), and the threats to validity (Chapter 6).

All the artifacts described here — the corpus generator, the generated corpus, the campaign runner, the raw execution data, and the analysis notebooks — are provided in the accompanying repository, whose layout and usage are summarized in [Appendix A](#).

Chapter 2

Background

2.1 FaaS composition and placement

Function-as-a-Service (FaaS) enables event-driven functions (f_i) to run without requiring developers to manage the underlying infrastructure [10]. Functions are generally stateless, so they access persistent data through external services (s_k), such as databases or APIs, and they are commonly orchestrated into compositions whose control and data flows make placement decisions interdependent.

A *placement* assigns every function to an execution location in the Cloud–Edge continuum. This entails three decisions: *local eligibility* determines whether one function can execute at one location; *global feasibility* verifies composition-wide constraints; and *preference optimization* ranks the feasible alternatives by considering QoS attributes such as cost, latency, and security. Approaches like SECFAAS2FOG [1] address the first two levels; CLASP addresses the third one without giving up the first two.

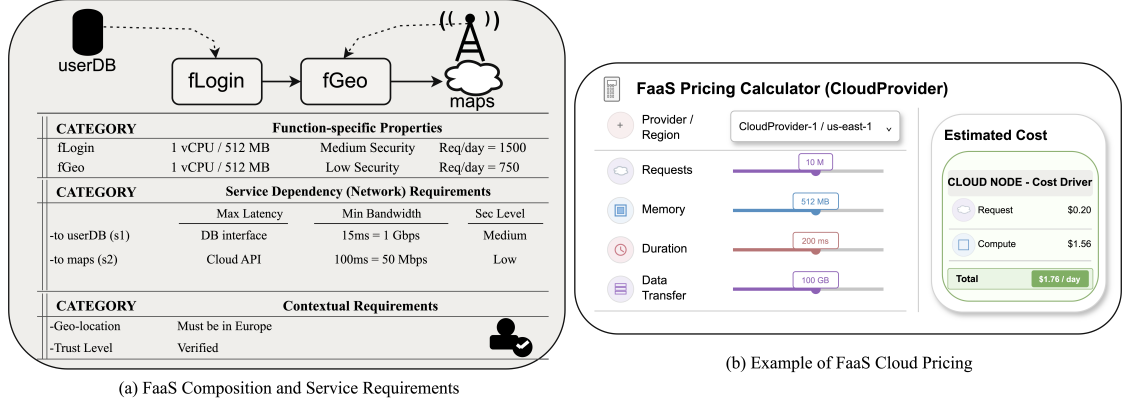
2.2 The binding problem

QoS-aware service composition (QACO) selects one concrete service candidate for each abstract task in a composite application, subject to constraints and optimization objectives [12, 3]. The resulting assignment is called a *binding*. A classic QACO problem instance can be described as $\mathcal{I} = (\mathcal{M}_A, \mathcal{M}_C, \Delta, O)$, where $\mathcal{M}_A = (T, G, \Lambda)$ represents the application model, $\mathcal{M}_C = (P, C, F)$ represents the candidate model, Δ captures the constraints, and O defines the optimization objective. Here, T contains the abstract tasks, G their orchestration workflow, Λ the aggregation policies for quality attributes, P the providers, C the candidate implementations, and F their local QoS attributes.

For each task t_i , let $C_i \subseteq C$ be the candidates available for that task. The *binding space* is the Cartesian product of those candidate sets, $\mathcal{B} = C_1 \times C_2 \times \dots \times C_m$: the search space before evaluating constraints that involve complete assignments. Local compatibility rules may reduce the candidate sets, whereas global constraints, aggregation policies, and objectives identify the feasible bindings and rank them. OpenBinding captures QACO instances through the Binding Instance Model (BIM), an engine-independent representation that separates the problem semantics from the solving engine [3].

2.3 SaaS and FaaS pricings

Pricings expose the configuration space of a service by organizing features and usage limits into plans and add-ons [9]. Pricing-driven DevOps proposes representing these structures



Placement Case		Feasibility Status	Detailed Feasibility Analysis	Est. Cost/month	Network Latency	Security Level	Trusted	Max Req/day
Case A: Cloud-Centric (Entirely on cloudNode)	 cloudNode	Eligible	Satisfies all CPU / RAM	\$420	near-zero (internal) 12ms (avg)	All	YES	1000000
Case B: Edge-Centric (fLogin on northAntenna, fGeo on cloudNode)	  northAntenna, cloudNode	Eligible	satisfies all security	\$65	12ms (avg)	Medium	YES	3000
Case C: Over-Provisioned Edge (Entirely on southAntenna)	 southAntenna	Non-Eligible	Non-Eligible (Failure to comply with constraints)	N/A - failure	N/A - failure	N/A - failure	N/A - failure	N/A - failure

(c) Example of Placement and Metric analysis

Figure 2.1: From FaaS requirements and workload-dependent cost estimation to feasible placement alternatives (values are illustrative). Feasibility separates admissible from inadmissible placements (placement C violates the constraints), but it cannot choose between the eligible placements A and B; that requires an explicit objective and comparable QoS estimates.

as machine-processable artifacts, known as *iPricings*, so that pricing information can be systematically analyzed and reused throughout the software lifecycle [4]. Pricing Intelligence broadens this perspective by encompassing the processes and tools that transform pricing data into actionable knowledge [2].

FaaS offerings differ from conventional subscription-based SaaS pricings because their charges are primarily determined by metered execution. The applicable rates depend on configuration variables, such as the provider, region, execution mode, architecture, and allocated resources, whereas the final charge depends on consumption variables, such as the number of invocations and execution duration. Providers express these combinations through billing meters — including requests, GB-seconds, vCPU-seconds, or GiB-seconds — whose unit prices may vary across regions and configurations. As illustrated by the calculator in Fig. 2.1, estimating and comparing candidate FaaS deployment costs therefore requires two complementary inputs: the provider’s billing rules and an expected-consumption profile that translates workload and resource estimates into the billing-meter quantities necessary to evaluate them.

Chapter 3

OpenBinding4Placement: Implementation

CLASP is the optimization problem of selecting, among the eligible placements of a FaaS composition, the binding that best satisfies a QoS-aware objective J and related deployment criteria. OpenBinding4Placement is the implementation that solves CLASP by translating it into QACO', representing it as BIM', and delegating the search to interchangeable engines.

Conceptually, a CLASP instance contains a FaaS composition, a Cloud–Edge infrastructure, external service-access requirements, function demands, node capacities, QoS-related requirements and information, external pricing or cost information, and an optimization objective. A placement assigns each function to one execution candidate; it is *eligible* if the complete binding satisfies all requirements, and it is *preferred* if it minimizes J among the eligible ones.

OpenBinding4Placement works as follows. (1) Expected-consumption estimates and iPricings are transformed into per-candidate cost attributes (Section 3.2). (2) The problem is encoded as a BIM' instance (Section 3.3). (3) The gateway validates the instance and routes it to a solving engine (Sections 3.1 and 3.4), and (4) re-evaluates every returned binding with a common reference evaluator that recomputes the objective value (Section 3.5). (5) Since solving is stateless and budget-bounded, the same endpoint is able to compute an initial binding b_0 and a replacement b_1 when conditions change.

3.1 Architecture

OpenBinding [3] is organized as an API gateway (a Python/FastAPI service) in front of a set of independently deployed solving engines, each running as its own service (Table 3.1). Clients submit a problem instance together with an `engine_id` to a single `/v1/solve` endpoint; the gateway validates the instance, forwards it to the selected engine, polls the job when the engine is asynchronous, and returns the solution enriched with the gateway's own feasibility verdict and canonical objective value.

Validation is a staged pipeline. *Stage 0* resolves the engine plugin in the registry. *Stage 1* validates the instance against the **general schema** — the engine-agnostic BIM structure. Instances that declare a `resource_model` or a `latency_model` are automatically detected as placement-aware and validated against the BIM' variant of the general schema instead (implementation slug: `bimstar`); BIM' is thus a general-schema variant, not an engine specialization. *Stage 2* validates the **specialization schema** of the selected engine (engine-specific capabilities and options). *Stage 3* runs general **semantic** checks (undefined tasks, invalid references, aggregation-policy consistency), and *Stage 4* delegates to the engine plugin's own

Table 3.1: Solving engines available behind the OpenBinding4Placement gateway. The three marked engines are the ones compared in the experimental campaign.

Engine id	Technology	Type	Role
minizinc-csp*	TypeScript + MiniZinc/Gecode	exact	Constraint-programming formulation able to prove optimality.
random-search*	Java	heuristic	Seeded uniform sampling of the binding space; baseline of the study.
evolutionary-heuristics*	Java + jMetal	heuristic	NSGA-II (mono-objective configuration in this study).
many-heuristic	Java	heuristic	Many-objective/Pareto engine; not part of the campaign.

semantic validation. A separate analysis step computes the binding-space cardinality and emits structured warnings.

3.2 From FaaS pricing to candidate costs

Provider-specific billing rules are represented as machine-readable data rather than embedded in the placement model or solver. We define FaaS-oriented Pricing2Yaml models — iPricings — for AWS Lambda, Azure Functions, and Google Cloud Run functions (directory `pricings/` in the repository). Because existing Pricing2Yaml versions support only inline scalar variables, they cannot adequately represent multidimensional FaaS pricings, whose applicable rates depend on region, architecture, billing meter, and resource class. We therefore introduce an experimental version of the syntax, whose metamodel is shown in Fig. 3.1, in which variables are typed declarative entities that can be resolved from external structured sources:

- **Billing modes** are modeled as *plans* (e.g., `CONSUMPTION` and `FLEX_CONSUMPTION` for Azure Functions; `REQUEST_BASED` and `REQUEST_BASED_WITH_MIN_INSTANCES` for Google Cloud Run functions).
- **Billable meters** (requests, GB-seconds, vCPU-seconds, GiB-seconds) are modeled as quantitative *add-ons*, and monthly free grants as *renewable usage limits*.
- **Selectors** such as region and architecture are *pricing variables* with `valueType: ENUM`, whose domains may be computed from other variables, e.g.
`enumValues: '#pricesPerRegionAndArchitecture.keys()'`.
- **Volatile rate tables** are variables with `valueType: EXTERNAL`, resolved through a *connector* (Listing 3.1). In this study the connector type is `LOCAL`: rates live in a sibling `*_variables.yml` file that is regenerated from the provider pricing APIs by the accompanying `generate*_variables.py` scripts, without modifying the stable structure of each offering.
- **Prices reference variables** through `#` expressions, e.g.
`price: '#pricesPerRegionAndArchitecture[#region][#architecture]["firstGbSeconds"]'`.

Listing 3.1: Excerpt of the Azure Functions iPricing (pricings/azure/azure_functions_pricing.yml): typed declarative variables of the experimental Pricing2Yaml version. The ENUM region selector derives its domain from an EXTERNAL rate table resolved through a LOCAL connector.

```

syntaxVersion: '3.2'
name: Azure Functions
currency: USD
variables:
  consumptionRegion:
    description: Azure region used for the legacy Consumption plan meters.
    defaultValue: eastus
    valueType: ENUM
    enumValues: '#consumptionPricesPerRegion.keys()'
  consumptionPricesPerRegion:
    description: Consumption plan execution and GB-second prices by region.
    valueType: EXTERNAL
    connector:
      type: LOCAL
      path: ./azure_functions_variables.yml
      key: consumptionPricesPerRegion

```

Table 3.2: On-demand monthly cost formulas evaluated by the metering transformation. Rates are read from the iPricing variables files and refreshed from the provider pricing APIs.

Provider	Monthly cost (on-demand)
AWS Lambda	$inv \times \text{lambdaRequest} + \text{GB-s} \times \text{x86.firstGbSeconds}$
Azure Functions	$inv \times \text{execution} + \text{GB-s} \times \text{gbSecond}$
Google Cloud Run functions	$inv \times \text{request} + \text{vCPU-s} \times \text{vcpuSecond} + \text{GiB-s} \times \text{gibSecond}$

3.3 QACO' and BIM': a placement-aware binding model

A QACO instance can be represented as $\mathcal{I} = (\mathcal{M}_A, \mathcal{M}_C, \Delta, O)$, where $\mathcal{M}_A = (T, G, \Lambda)$ captures the application and $\mathcal{M}_C = (P, C, F)$ captures the candidates and their QoS features (Chapter 2). However, CLASP also depends on the deployment infrastructure: candidates may share node resources, and communication latency depends on the nodes selected by the binding.

We therefore define QACO', a placement-aware extension of QACO. Its candidate model is extended as $\mathcal{M}'_C = (P, C, F, R, L)$, where R describes the available node resources and L the network latencies. A QACO' instance is consequently defined as $\mathcal{I}' = (\mathcal{M}_A, \mathcal{M}'_C, \Delta, O)$. QACO' defines the CLASP optimization problem and its semantics; to exchange instances of this problem independently of any particular solving technology, OpenBinding4Placement represents them using BIM', the placement-aware extension of the Binding Instance Model: it serializes the elements of a QACO' instance, including its resource-capacity and network-latency information, in a solver-independent format.

The mapping from CLASP to QACO' is as follows. Each function f_i is represented by a task t_i , and each feasible deployment of that function on a node n_j is represented by a candidate c_{ij} . When an external service s_k must also be selected, the candidate c_{ijk} represents both decisions (a candidate may bundle deployment decisions, such as the set of selected services or the memory tier of a cloud function). A candidate is generated only when the node and the selected service satisfy the corresponding function requirements:

$$c_{ijk} \in C_i \iff req(f_i) \preceq cap(n_j) \wedge svc(s_k) \models svcReq(f_i). \quad (3.1)$$

Thus, requirements local to a single deployment decision are either used to filter the candidate sets or encoded as local constraints in Δ , whereas requirements involving several placement decisions are encoded as global constraints. A binding $b : T \rightarrow C$ directly determines the placement, since selecting c_{ijk} for t_i means deploying f_i on n_j and binding it to s_k . Consequently, every feasible QACO' binding is an eligible placement, while the objective selects the preferred one.

3.3.1 The BIM' serialization

Concretely, a BIM' instance is a JSON document that extends the general BIM sections (`metadata`, `tasks`, `providers`, `candidates`, `composition`, `aggregation_policies`, `constraints`, `objective`) with two placement sections (Listing 3.2):

- **resource_model** — the declared resource kinds, the infrastructure *pools* (nodes) with their capacities, the pool binding and resource demands of every candidate, and **RESOURCE_CAPACITY** dependency constraints: the cumulative demand of the candidates placed on a pool must not exceed its capacity, for every resource (a bin-packing condition per node).
- **latency_model** — a pool-to-pool network latency matrix, event-generator pools with their own latency rows, pairwise *transition constraints* bounding the latency between the pools hosting two related tasks (or an event generator and a task), and the declaration of the end-to-end latency attribute.

Additionally, BIM' gives the composition-level semantics needed for placement: the end-to-end latency of a binding is defined as the *expected makespan over the XOR scenarios* of the composition (each scenario fixes one branch per XOR node, turning the composition tree into a precedence DAG scheduled by critical path, with network transfer delays between the pools of consecutive tasks); **SAME_POOL/DIFFERENT_POOL** dependency constraints

Listing 3.2: Fragment of a real corpus instance (`stockOrch`, 50-node infrastructure): the two placement sections that BIM' adds to the general BIM, and a priced cloud candidate.

```
"resource_model": {
  "resources": ["memory_mb", "vcpu", "mhz", "concurrency"],
  "pools": [
    {"id": "edge1", "kind": "EDGE",
     "capacity": {"memory_mb": 1024.0, "vcpu": 4.0, "mhz": 2500.0},
     "metadata": {"provider": "privateCitizen1", "node_type": "edge"}},
    ...],
  "candidate_bindings": {"cand_fCheck_edge4_orig_9f8d939d": {
    "pool": "edge4", "demand": {"memory_mb": 512.0, "vcpu": 1.0}}, ...},
  "constraints": [{"kind": "DEPENDENCY", "type": "RESOURCE_CAPACITY", ...}]
},
"latency_model": {
  "unit": "ms",
  "pool_latency_matrix_ms": {"edge1": {"edge1": 0.0, "cloud1": 37.0,
                                         "fog1": 8.0, ...}, ...},
  "event_generator_pools": [...],
  "transition_constraints": [...],
  "global_latency": {...}
},
"candidates": [
  {"id": "cand_fCheck_edge4_orig_9f8d939d",
   "task_id": "fCheck", "provider_id": "privateCitizen2",
   "features": {"latency": 0.0, "cost": 12.545, "security": 0.66,
                "has.service.database": 1.0, ...}},
  ...]
```

express co-location and separation requirements; and *canonical normalization bounds* are embedded per feature in the aggregation policies, so that every engine optimizes and reports the same normalized weighted objective $J \in [0, 1]$. The BIM' JSON Schema lives in `schemas/general/bimstar.schema.json` and every generated instance is validated against it.

3.4 Solving engines

Three heterogeneous engines were extended with a placement-native path while keeping their models untouched from the client's perspective; the same BIM' instance is submitted unmodified to any of them.

Exact constraint programming (minizinc-csp). A TypeScript service translates each BIM/BIM' instance into MiniZinc data and solves it with Gecode [11]. The placement extensions are encoded natively: cumulative resource capacities per pool, transition-latency bounds, and end-to-end latency as expected makespan scheduling over the XOR scenarios. The engine runs with `intermediate_solutions` enabled, reporting every incumbent it finds together with its timestamp, and terminates with a completion status: `OPTIMAL` (optimality proven), `SATISFIED` (feasible incumbent, budget exhausted before the proof), `UNSATISFIABLE`, or `UNKNOWN`.

NSGA-II (evolutionary-heuristics). A Java service built on jMetal runs NSGA-II [6] in mono-objective mode (`MONO`): a single scalarized fitness makes its results directly comparable, under the same objective, with the other engines. Feasibility is folded into the objective

via Deb’s rules [5], and variation uses categorical operators suited to unordered candidate indices — uniform crossover plus random-reset mutation — with a population of 20 individuals. This configuration was selected in a preliminary three-way pilot on the hardest application (Section 4.4). The engine tracks the best individual ever evaluated, giving it anytime behavior.

Random search (`random-search`). A Java service that repeatedly samples placements uniformly at random (seeded, hence reproducible), keeping the best sample. It provides the baseline that mimics repeatedly drawing valid placements without an advanced optimization strategy.

Anytime protocol. All engines honour a shared wall-clock budget (`time_budget_ms`), return their best-so-far incumbent on timeout, and emit an improvement trace with one (`eval_index`, `elapsed_ms`) event per improvement. This enables the offline cutoff analyses of Chapter 5 and matches the operational needs of self-adaptation, where a rebinding decision must be available whenever the environment demands it.

3.5 The reference evaluator

The **reference evaluator** (`openbinding_gateway.validation.engine_plugins.bimstar`) is the single source of truth for the placement semantics: end-to-end latency as expected makespan over the XOR scenarios, resource-capacity and transition constraints, `SAME_POOL` / `DIFFERENT_POOL` dependencies, and the canonical normalized objective. Every solution returned by any engine is re-evaluated with it, which underpins the fair-comparison protocol of the study:

1. **One referee** — only the canonical J recomputed by the reference evaluator reaches the results dataset and the evaluation; solver-reported numbers are never compared directly.
2. **Same landscape, proven** — the normalization bounds are part of the instance (a declared contract), all engines use the same weighted-mean-of-losses convention, and each run carries an integrity audit (`oracle_match`): the engine’s internal objective must equal the canonical J for feasible solutions.
3. **Scale-free aggregation** — J is normalized per instance, so raw values are only comparable within an instance; cross-instance conclusions use gaps to the per-instance reference, improvement over the baseline, Dolan–Moré performance profiles [7], and mean ranks with Friedman and per-instance Mann–Whitney/ $\hat{A}_{12}$ tests.
4. **Feasibility first** — infeasible solutions are never compared by J ; they are reported through success rates and violation magnitudes.

Chapter 4

Experimental Design

4.1 Research questions

The evaluation addresses four aspects of OpenBinding4Placement:

- **RQ₁ — Feasibility versus optimization.** How much placement quality is forgone when the search stops after finding a feasible placement?
- **RQ₂ — Quality, runtime, and scalability.** Which solving strategies provide the most appropriate trade-off between quality, runtime, and scalability?
- **RQ₃ — The time cost of optimization.** How much planning time does the optimization require, and when is it worth continuing the search?
- **RQ₄ — Suitability for runtime rebinding.** Can the evaluated engines provide solutions quickly enough to support runtime rebinding?

4.2 Source dataset

The benchmark derives from the dataset published with SECFAAS2FOG [1]. It contains three FaaS compositions — an augmented-reality application (`arOrch`, 7 functions, the hardest one), a media-processing pipeline (`mediaOrch`, 5 functions), and a stock-market application (`stockOrch`, 5 functions) — together with 20 randomly generated Cloud-Edge infrastructures per size, each available in 35 sizes from 50 to 220 nodes (step 5). Every function carries software and hardware requirements and external service dependencies; every node offers capacities, hosted services, security capabilities, and a node-to-node latency matrix. The campaign uses one dataset seed (146588263), i.e., one infrastructure per size.

4.3 Corpus generation: from SecFaaS2Fog to priced BIM' instances

A deterministic generator (package `experimentation/icsoc/bimstar/`, generator seed 12345) transforms each application $\times$ infrastructure pair into a priced BIM' instance. Notebook `01_dataset_preprocessing.ipynb` documents and executes the transformation; its main steps are:

1. **Tasks and composition.** Orchestration functions become BIM tasks; the orchestration grammar (`seq/par/if`) becomes a structured SEQ/AND/XOR tree (IF guards $\rightarrow$ XOR with $p = 0.5$ per branch).

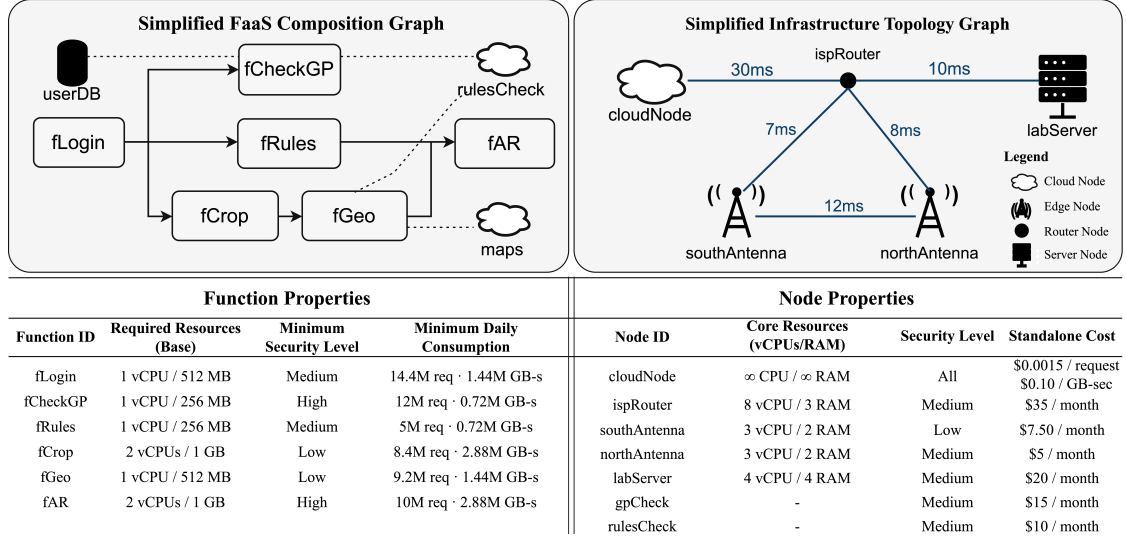


Figure 4.1: The augmented-reality composition of the dataset (left) and a fragment of a candidate Cloud-Edge infrastructure (right): the paper’s motivating application, and the hardest of the three compositions in the corpus.

- Candidates.** One candidate per (function, hosting option): the original edge/fog/-cloud nodes that satisfy the function’s software/hardware requirements and can bind its required services, plus *generated cloud FaaS candidates* for every provider $\times$ region $\times$ memory size $\geq$ the requirement.
- Pricing.** Each candidate receives an expected monthly USD cost computed from the iPricings of Section 3.2, using the on-demand plans, driven by the decision variables memory size (128–4096 MB), invocations per month, average duration, vCPUs, and region. At the campaign workload (1M invocations/month $\times$ 120 ms), the AWS compute volume (~ 0.5 M GB-s) is far below the 6B GB-s tier-1 limit, so the tier-1 rate is exact. Fig. 4.2 audits the resulting cost surface, and a worked example in the notebook recomputes one cost by hand from the raw iPricing rates and checks that it matches `FaaSPriming.estimate` (the same checks run in the unit tests).
- Latency model.** Original node-to-node latencies are kept; pairs involving generated cloud pools use a provider- and geography-aware model with *disjoint* ranges per class, so that links between regions of the same provider (private backbone) are faster than links between different providers (public peering) in the same geography class, and intra-geography links are faster than inter-geography ones (Fig. 4.3).
- Security.** Information-flow labels are propagated over the orchestration (SECFAAS2FOG typing) with per-variable dataflow: trigger labels ride on the named dataflow variables, an output named like an existing variable keeps that variable’s label, and only genuinely new variables inherit the task’s level. Per-task minimum security thresholds therefore vary (e.g., in `arOrch`: `fLogin` 1.0 but `fCrop/fGeo` 0.66), so the security term of the objective genuinely discriminates instead of saturating. Node security capabilities map to scores (none = 0.33, `pubKeyE` = 0.66, `pubKeyE+antiTamp` = 1.0).
- Budgets and canonical bounds.** Eligible candidate sets come from latency-arc-consistent (AC-3) pool domains filtered by the per-task security thresholds. Since AC-3 only guarantees arc consistency, budgets are anchored to a *certified feasible witness*: a cheapest-first backtracking assignment over the filtered domains that satisfies every

Table 4.1: Corpus characterization per application (35 instances each; dataset seed 146588263). The binding space is the product of per-task candidate counts.

Application	Tasks	Candidates	Pools	$\log_{10}$ binding space
ar0rch	7	474–2 314	56–226	12.4–16.2
media0rch	5	496–1 613	56–226	9.8–12.2
stock0rch	5	330–1 259	56–226	8.8–11.5

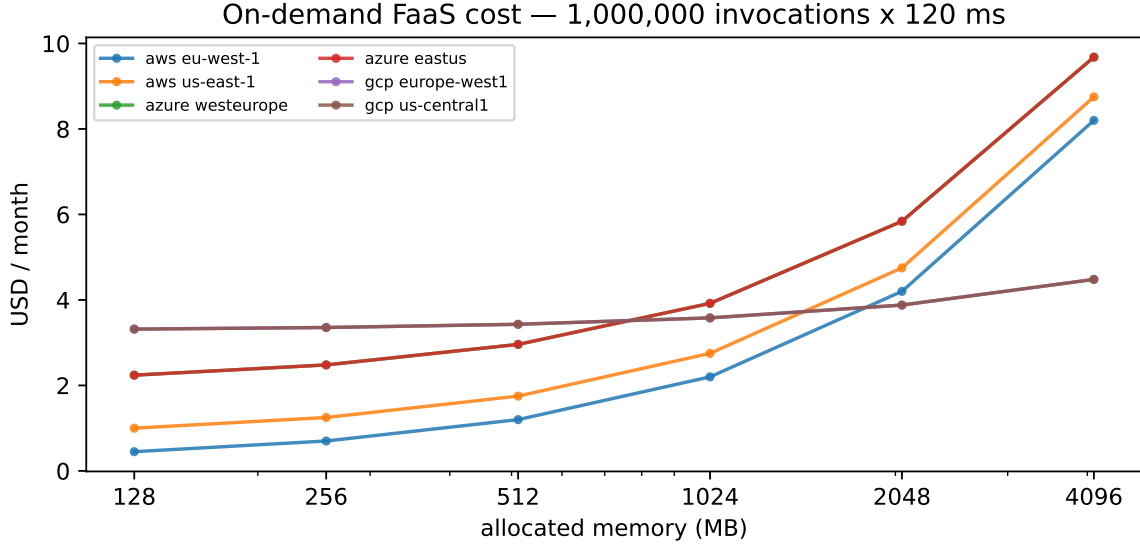


Figure 4.2: Pricing audit: monthly on-demand cost of generated cloud candidates as a function of allocated memory, per provider, at the campaign workload. The disjoint provider bands follow directly from the iPricing rates.

transition bound and pool capacity. Per-task budgets are the maximum of the 75th percentile of eligible costs and the witness cost; the global budget is the witness cost plus a configurable slack of the headroom up to the fold of the local budgets (Fig. 4.4). Every instance is therefore satisfiable by construction, with configurable difficulty. Finally, per-feature min-max normalization bounds, shared by every engine, are embedded in the instance.

The result is a corpus of **105 schema-valid BIM' instances** (3 applications $\times$ 35 sizes $\times$ 1 dataset seed) plus full provenance traces (per-candidate pricing, latency, security, and budget-derivation reports as CSV files). Table 4.1 and Fig. 4.5 characterize the corpus: candidate sets grow to over two thousand candidates per task set and binding spaces reach 10^{16} combinations.

4.4 Campaign protocol

The campaign (Table 4.2) is executed by a resumable runner (`experimentation/icsoc/campaign.py`) that submits every (instance, engine, seed) combination to the gateway's `/v1/solve` endpoint, polls the job, re-evaluates the returned binding with the reference evaluator, and appends one row per run to `runs.csv` and one row per improvement event to `traces.csv`. Run ids are deterministic (`instance|engine#seed`), so interrupted lanes resume where they stopped, and the campaign can be sharded by application, size, or engine

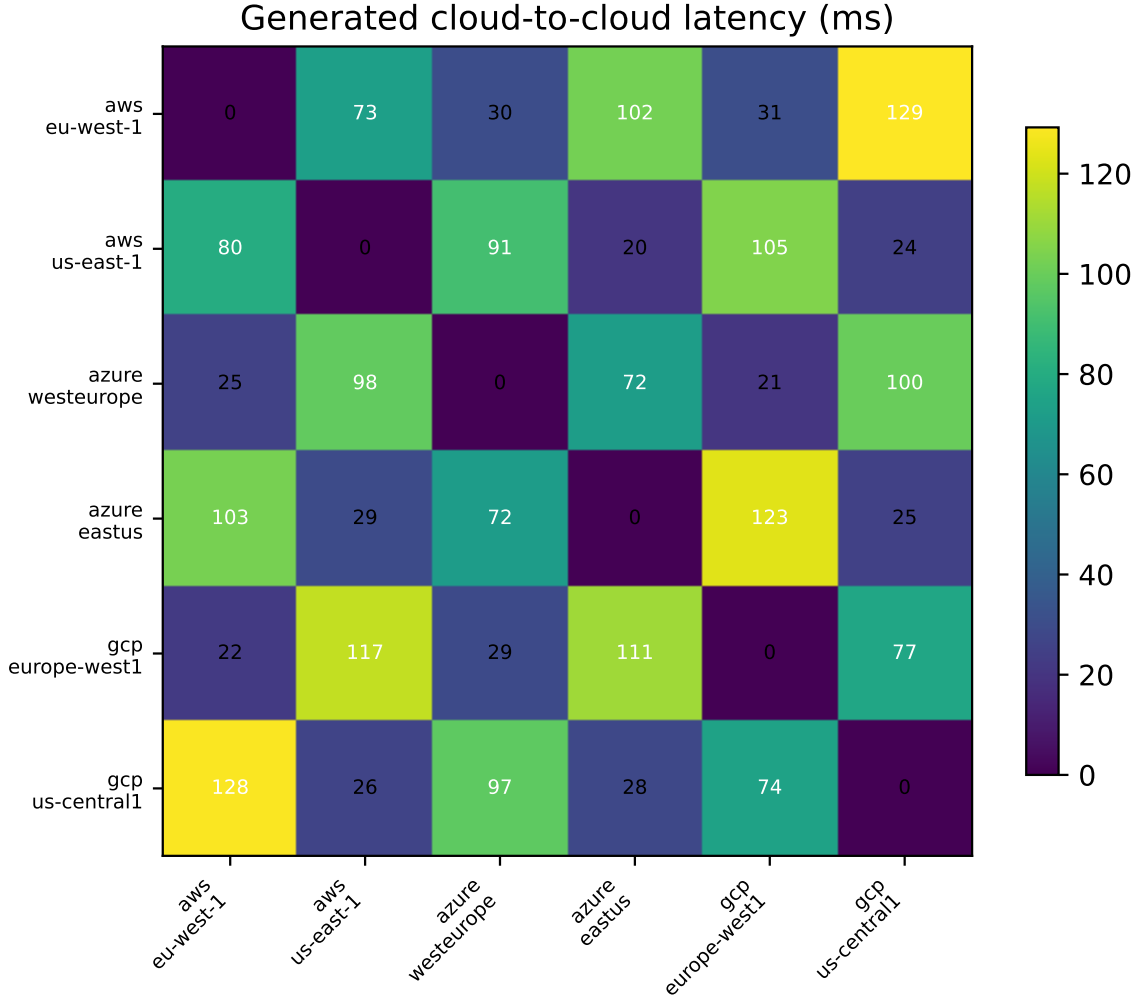


Figure 4.3: Latency audit: pool-to-pool latency matrix of a generated instance. Generated cloud-pool pairs follow the provider/geography class model with disjoint ranges; original dataset latencies are preserved.

Table 4.2: Experimental design of the campaign.

Variable	Domain
Algorithms	exact (MiniZinc/Gecode), random search (baseline), NSGA-II (pop. 20, uniform crossover + random-reset mutation)
Objective	$J = 0.34 \cdot \text{loss}(\text{cost}) + 0.33 \cdot \text{loss}(\text{latency}) + 0.33 \cdot \text{loss}(\text{security})$, canonical, lower is better
Instances	3 applications $\times$ 35 infrastructure sizes (50–220 nodes) $\times$ dataset seed 146588263 = 105
Repetitions	exact $\times 1$; stochastic $\times 10$ (solver seeds 1–10) $\Rightarrow$ 2 205 runs
Stopping criterion	shared wall-clock budget $T = 300$ s for every algorithm
Heuristic floor	$\geq 1\,000$ evaluations always granted (standard literature budget)

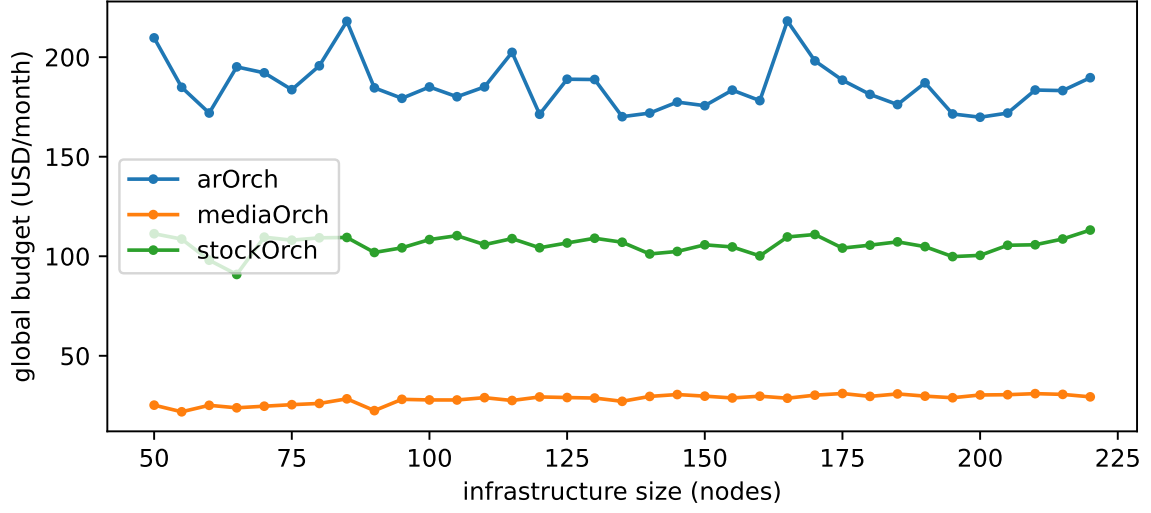


Figure 4.4: Budget derivation: certified witness cost and derived global budget across infrastructure sizes, per application. Budgets are anchored to a feasible witness, so every instance is satisfiable by construction.

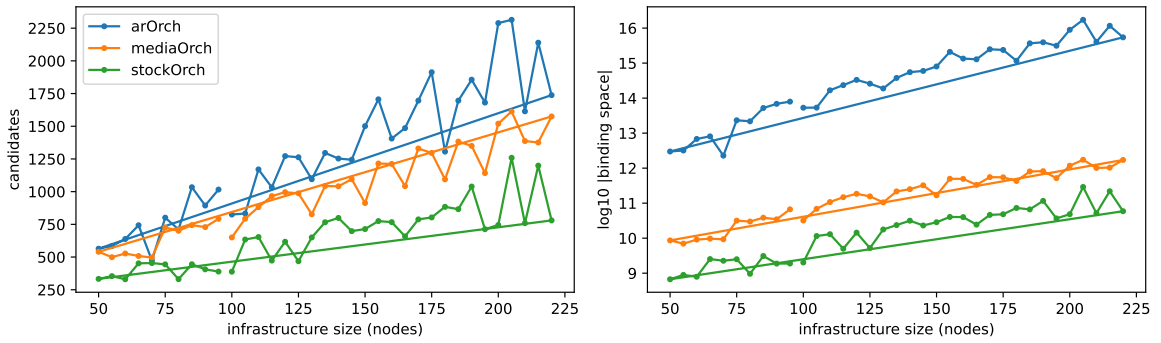


Figure 4.5: Corpus characterization: candidates, pools, constraints, and binding-space size across the 105 instances — the covariates used in the evaluation.

Table 4.3: GA pilot on `arOrch` (3 instances $\times$ 3 seeds per configuration, budget $T = 300$ s).

Configuration	Feasible runs	Median J	Median evaluations
pop. 20, SBX + polynomial	7/9	0.154	1.44M
pop. 50, SBX + polynomial	8/9	0.106	1.19M
pop. 20, uniform + random reset	9/9	0.104	1.24M

across machines. The recommended execution is three parallel engine lanes; each lane is sequential inside its engine container, which keeps the stochastic runs reproducible (one JVM-global RNG per engine).

Engine configurations. The exact engine receives `time_limit_ms = T` with intermediate solutions enabled. Both stochastic engines receive `time_budget_ms = T`, a guaranteed floor of 1 000 evaluations, and their seed. NSGA-II runs with `algorithm: NSGAI`, `operators: UNIFORM`, and `population_size: 20`.

GA configuration pilot. The NSGA-II configuration was selected in a preliminary three-way pilot (`pilot_ga.py`) on the hardest application (`arOrch`), comparing three configurations under the campaign budget on three instances $\times$ three seeds (Table 4.3). Uniform crossover with random-reset mutation (the standard variation for categorical candidate indices) achieved full feasibility and the best median objective with the smaller population, and was clearly better at the classic 1 000-evaluation cutoff; SBX-based variation — designed for continuous representations — lost feasibility in two of nine runs.

Reference per instance. The reference for each instance is the proven optimum when the exact search completes (`OPTIMAL`), or otherwise its best feasible anytime incumbent (`SATISFIED`); if the exact engine returned nothing, the best heuristic value at the standard 1 000-evaluation cutoff would be used as fallback (derived offline from the traces; this fallback was never needed in the campaign).

Execution environment. The experiments ran without GPU resources on a macOS laptop (Intel Core i9-9880H, 16 GB RAM), with the gateway and the engines deployed as Docker Compose services. Engine solver time is recorded separately from gateway overhead (`engine_execution_time_ms`), and all statistical analyses use the isolated solver time.

Chapter 5

Evaluation

This chapter reports the full evaluation of the campaign, following the four research questions of Section 4.1. All figures are produced by the analysis notebook (`03_results_evaluation.ipynb`) from the raw `runs.csv` and `traces.csv` data; the paper presents a subset of them, while this chapter includes the complete set. Throughout the chapter, J denotes the canonical normalized objective (lower is better) recomputed by the reference evaluator, and statistical comparisons use paired, scale-independent measures: relative optimality gaps, Dolan–Moré performance profiles [7], Friedman’s test and mean ranks [8], and Mann–Whitney U tests with Vargha–Delaney $\hat{A}_{12}$ effect sizes [13].

5.1 Technical validation

All 105 instances were serialized as BIM’ and processed in all 2 205 submissions without errors, confirming that the same solver-independent representation supports the three solving strategies. The reference evaluator found no implementation-level discrepancy: engine-reported and canonical objective values differed by at most 10^{-15} (the `oracle_match` audit reported zero mismatches over the 2 205 runs), and no heuristic outperformed a proven optimum.

Exact solving and NSGA-II returned feasible bindings in every run; random search did so in 98.9% of its runs (Fig. 5.1). The exact engine proved optimality for 89 instances (**OPTIMAL**) and returned a feasible anytime incumbent for the remaining 16 (**SATISFIED**); the heuristic 1 000-evaluation fallback reference was never needed. These results demonstrate coverage of the benchmark and consistent interpretation of feasibility and quality across the three extended engines.

5.2 RQ₁: Feasibility versus optimization

To quantify the distinction between eligible bindings, we compare the first feasible placement produced in each run with the best placement obtained before the time limit. The first solution approximates approaches that stop once eligibility is established; although this does not reproduce the internal selection policy of [1], it enables a controlled comparison using the same instance.

Continuing the search substantially improved placement quality, reducing the median objective by 68% for NSGA-II (from $J = 0.29$ for the first feasible binding to $J = 0.09$ for the final one) and 63% for random search (Fig. 5.2). Both engines initially produced placements with a median gap of approximately 270% above the proven optimum, but their final results differed considerably: NSGA-II reached the optimum in half of its runs and

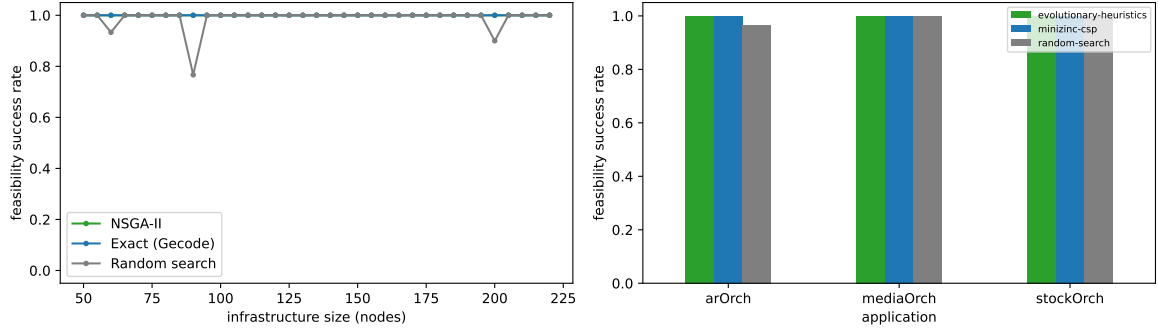


Figure 5.1: Feasibility (success) rates per engine across infrastructure sizes and per application. Success = the returned solution satisfies all hard constraints according to the reference evaluator.

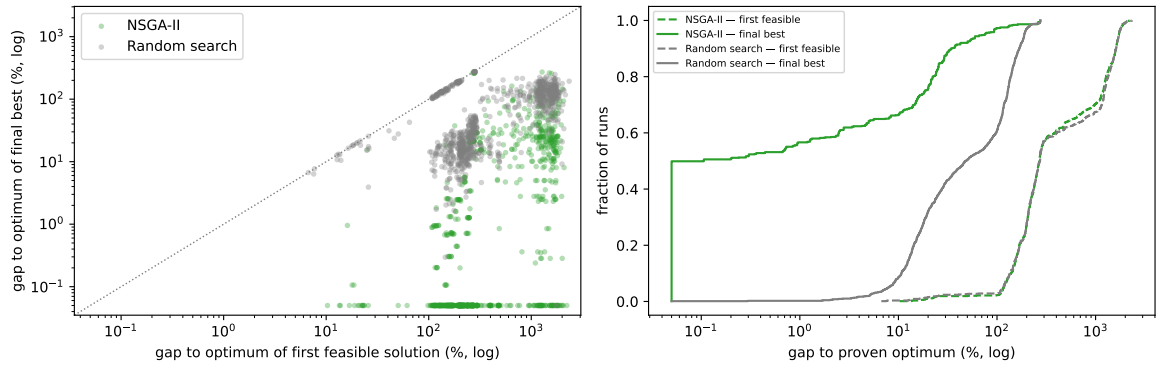


Figure 5.2: First feasible placement vs. best placement obtained in the same run. Left: per-run gap to the proven optimum of the first feasible solution vs. the final best. Right: fraction of runs below each gap, before (dashed) and after (solid) continuing the search. The difference quantifies the quality forgone when placement stops at validity.

obtained a median gap of 0.1%, whereas random search remained 48% above it in the median (Figs. 5.3 and 5.4).

The exact engine and NSGA-II also consistently improved upon the random-search baseline, with larger gains for some infrastructure sizes (Fig. 5.5). The distribution of raw objectives per application (Fig. 5.6) and the median objective heatmap across engine and infrastructure size (Fig. 5.7) give the descriptive picture behind these paired statistics; note that raw J values are only comparable within an instance, so every cross-instance conclusion relies on the scale-free measures above.

These results show that feasibility alone does not indicate the relative quality of a placement: optimization is necessary to select among eligible bindings according to their combined cost, latency, and security attributes. These gains, however, come at a computational time cost (RQ₃, Section 5.4).

Answers to **RQ₁**: How much placement quality is forgone when the search stops after finding a feasible placement?

- (1) Eligibility alone cannot distinguish among valid placements.
- (2) Continuing the search substantially improves the balance among cost, latency, and security without modifying the feasible space (median objective reductions of 63–68%).

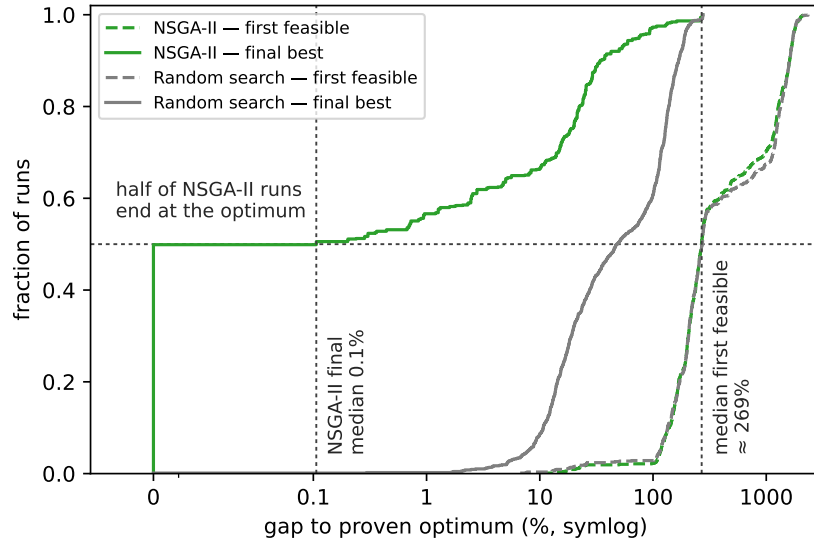


Figure 5.3: ECDF of the relative gap to the proven optimum for the first feasible (dashed) and final (solid) placements of the stochastic engines; vertical lines mark medians.

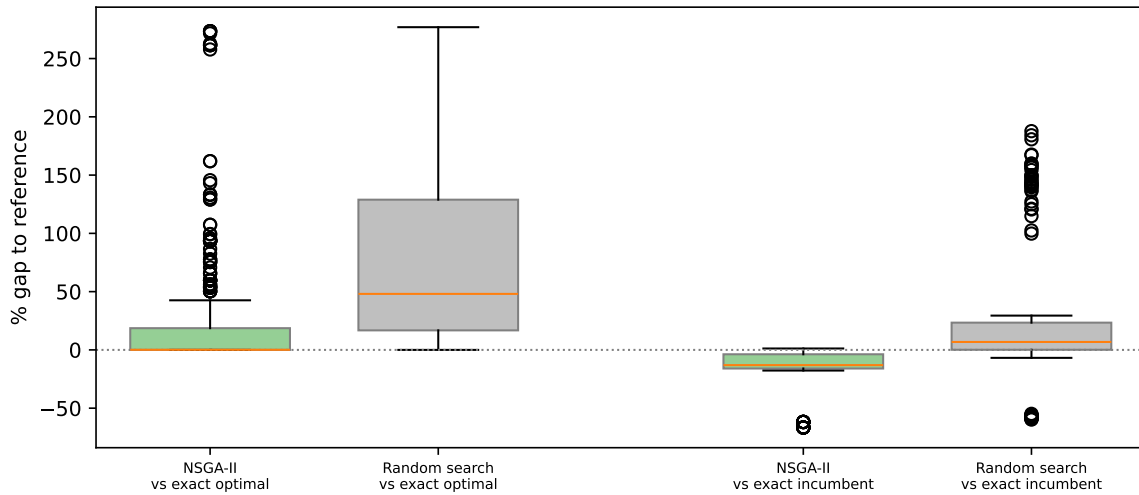


Figure 5.4: Final gap to the per-instance reference, split by reference source (proven optimum or exact anytime incumbent). Negative gaps against `exact_incumbent` mean the heuristic beat the exact solver's anytime solution within the same budget.

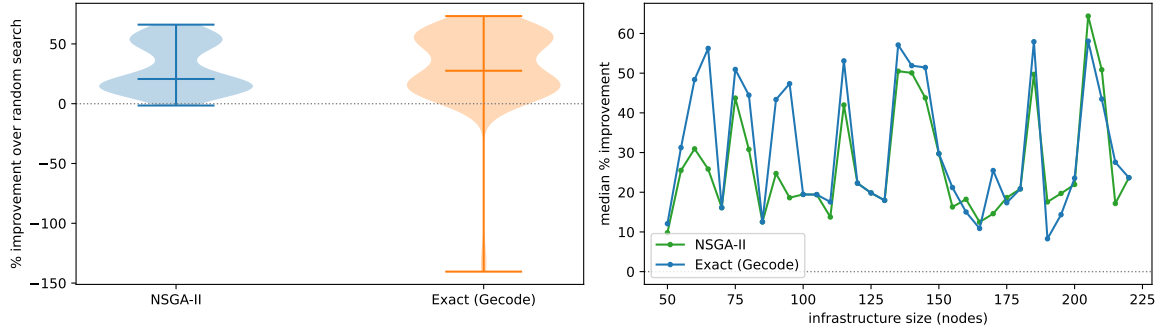


Figure 5.5: Relative improvement over the random-search baseline on per-instance medians: distribution per engine (left) and median by infrastructure size (right); positive means better than the baseline.

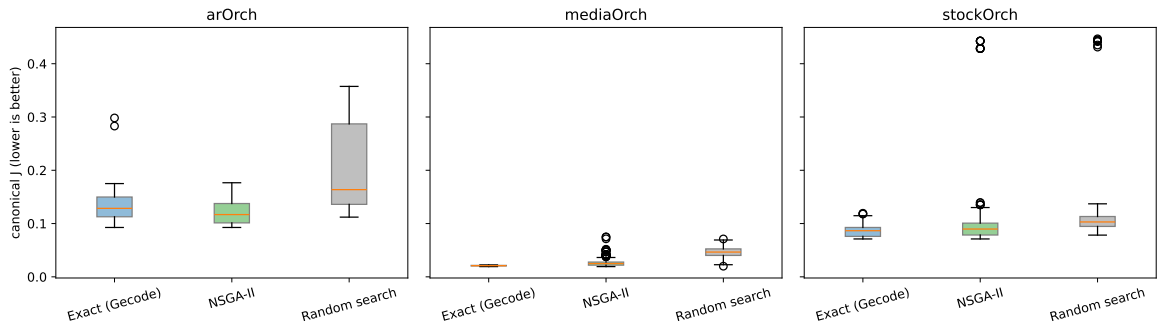


Figure 5.6: Distributions of the canonical objective J per engine and application (descriptive, per-application views; per-instance normalization applies).

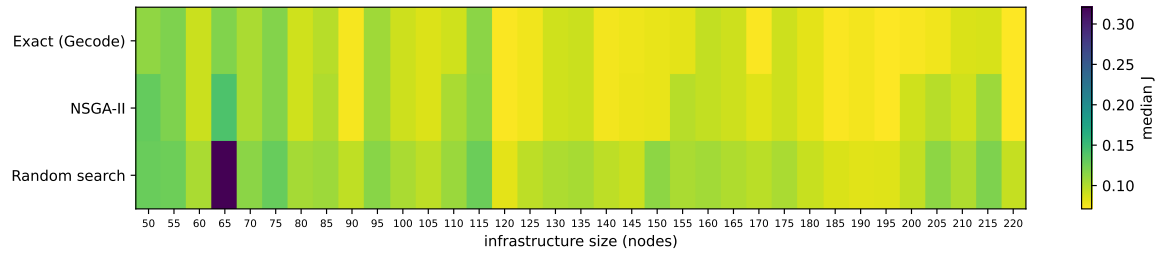


Figure 5.7: Median canonical objective per engine and infrastructure size.

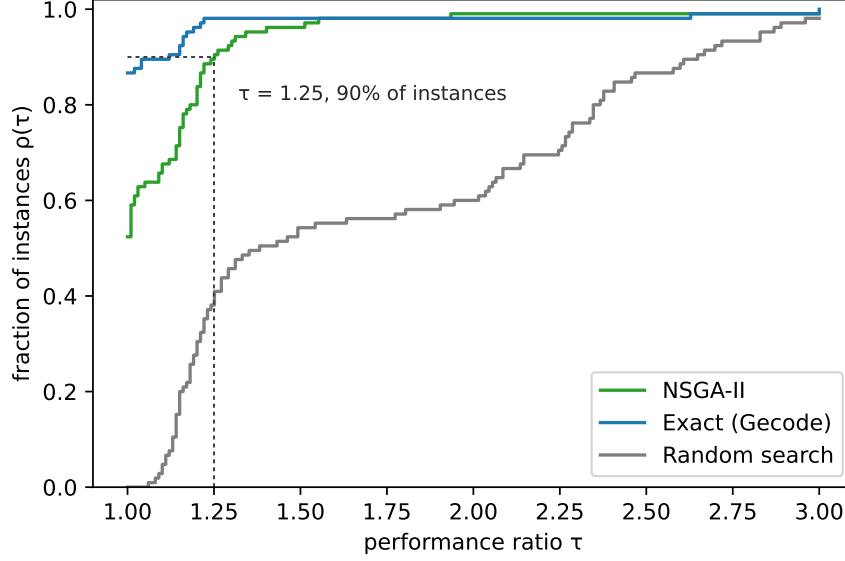


Figure 5.8: Dolan–Moré performance profiles over the 105 instances: fraction of instances whose engine median objective is within a factor τ of the best median on that instance. The dashed line marks NSGA-II coverage at $\tau = 1.25$.

5.3 RQ₂: Quality, runtime, and scalability

Fig. 5.8 compares final solution quality through Dolan–Moré performance profiles. Under the common 300-second budget, the exact engine produced a best-observed solution for approximately 87% of the instances. NSGA-II achieved the best result for about 52% and remained within a factor of 1.25 of it for roughly 90% of the benchmark, whereas random search required substantially larger factors to obtain similar coverage.

The paired statistical analysis confirms the superiority of NSGA-II over random search: NSGA-II won on 101 instances and tied on the remaining four, with no losses and a median $\hat{A}_{12} = 1.0$ (Fig. 5.9). This dominance is not an artifact of the time budget: at the classic 1 000-evaluation cutoff, NSGA-II already outperformed random search on all 105 instances (median gap to the reference of 105% vs. 401%), and both engines completed a similar number of evaluations within the same budget (about 2.5 million). Friedman’s test also rejected the hypothesis that the three engines perform equivalently ($\chi^2 = 164.06$, $p < 10^{-35}$), with mean ranks of 1.37 for the exact engine, 1.68 for NSGA-II, and 2.95 for random search.

Runtime behavior separates the strategies further (Fig. 5.10). Heuristic lanes always consume the shared budget; the exact engine terminates early when it proves optimality (median solver time 12.5 s), but its ability to prove optimality decreased with infrastructure size: all 16 instances without an optimality certificate involved the AR composition and, except for one case, infrastructures of at least 150 nodes. On these instances, NSGA-II returned objectives that were 13% lower in the median than the exact engine’s incumbents. This does not prove that the heuristic solutions were optimal, but it demonstrates that proof-oriented search and incumbent quality behave differently under a fixed time limit.

The results therefore support complementary uses for exact and heuristic solving: exact search is preferable when an optimality certificate can be obtained within the available budget, whereas NSGA-II offers a more reliable quality–runtime trade-off for larger or time-constrained placement problems.

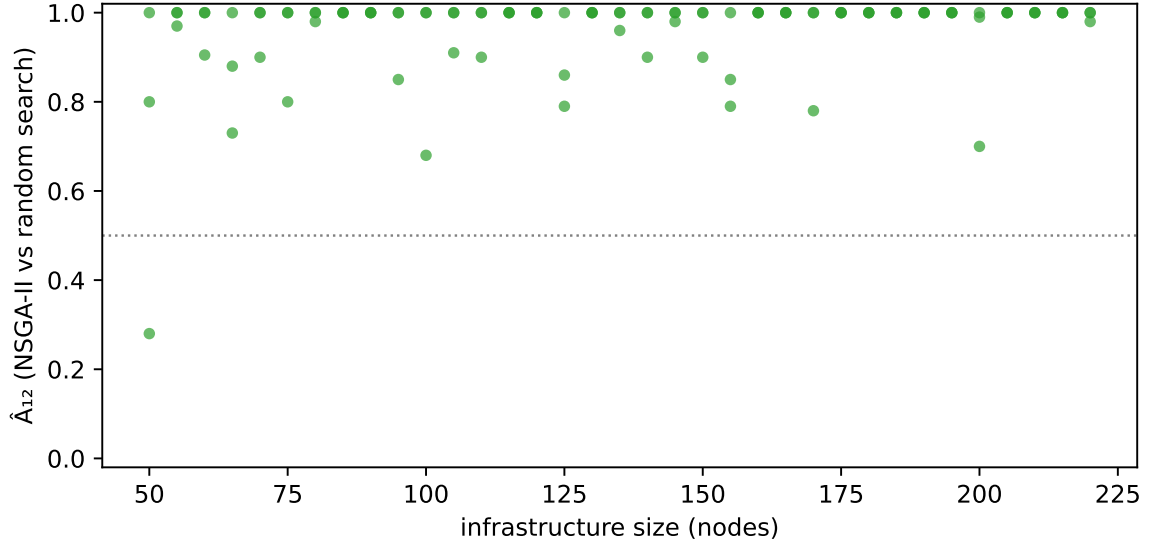


Figure 5.9: Per-instance statistical comparison of the stochastic engines (10 seeds per instance): Vargha–Delaney $\hat{A}_{12}$ effect sizes and win/tie/loss counts at $\alpha = 0.05$ (Mann–Whitney U).

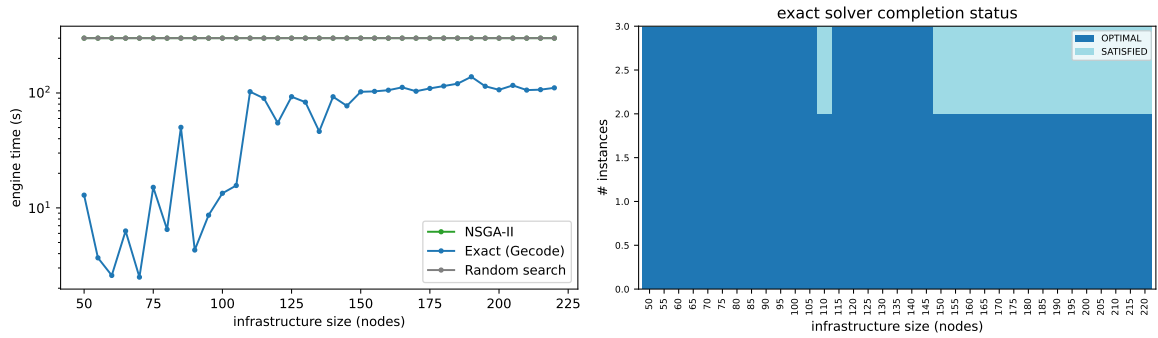


Figure 5.10: Engine solver time across infrastructure sizes and exact-engine completion status (OPTIMAL vs. SATISFIED). The exact engine finishes early when the proof succeeds and saturates the budget otherwise.

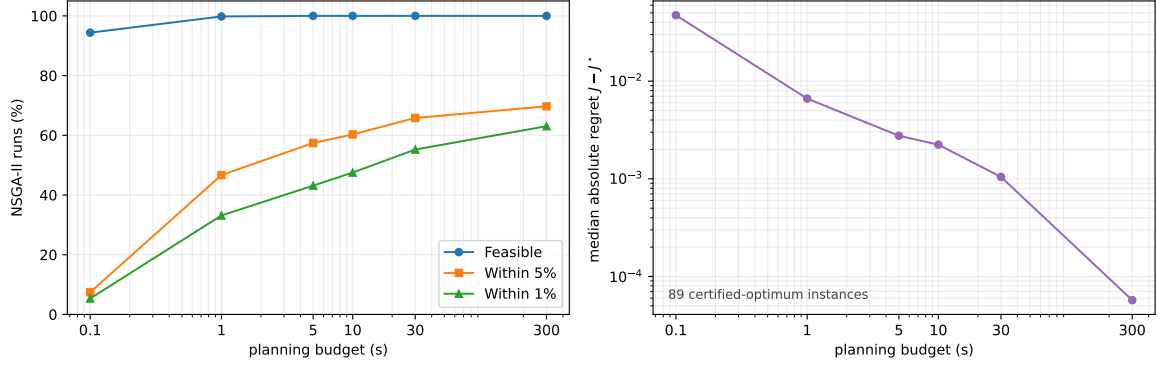


Figure 5.11: NSGA-II quality–deadline curves. Left: fraction of the 1050 runs that are feasible and within 5%/1% of the per-instance reference at each planning budget τ . Right: median absolute regret $J - J^*$, restricted to the 89 certified-optimum instances.

Answers to **RQ₂**: Which solving strategies provide the most appropriate trade-off between quality, runtime, and scalability?

- (1) Exact solving is preferred for smaller instances, when the available budget guarantees an optimal solution.
- (2) NSGA-II provides a more stable quality–runtime compromise for larger instances and time-bounded decisions.

5.4 RQ₃: The time cost of optimization

RQ₁ and RQ₂ measured the benefit of optimization after exhausting the 300-second budget, which raises the question any time-constrained deployment must face: how much of that time is actually needed, and what is lost by stopping earlier? Fig. 5.11 answers it by truncating every NSGA-II run at increasing planning budgets, using the recorded improvement traces.

The two extremes of the trade-off are clearly separated. Stopping at the first feasible placement is almost immediate — at a 0.1-second deadline, 94% of the runs already hold a valid binding — but only 5% of them are within 1% of the reference, reflecting the median quality cost of 68% quantified in RQ₁. A single additional second changes the picture: feasibility reaches 100%, a third of the runs are within 1% of the reference (47% within 5%), and the median regret on the 89 certified optima drops by an order of magnitude, from 4.7×10^{-2} to 6.6×10^{-3} . Beyond that point returns diminish gradually: each tenfold budget increase buys roughly five to ten additional points of coverage, up to 63% of runs within 1% at the full budget. Consistently, the median time of NSGA-II’s last improvement was 1.3 seconds (Fig. 5.12).

These gains stem from the search strategy, not from merely spending time. Random search produced its last improvement after a median of 143 seconds and still ended 48% above the optimum (RQ₁); at the 1000-evaluation cutoff, NSGA-II’s median gap to the reference was already 105%, against 401% for random search (Fig. 5.13). Both heuristics completed a similar number of evaluations within the budget (Fig. 5.14), and the time to reach the first feasible and the best solution is summarized in Fig. 5.15. The exact engine, in turn, buys optimality certificates at the price of the full budget on the largest instances, so exhausting the budget is only justified when a certificate is required (RQ₂).

Therefore, even deployments with tight planning budgets should not settle for the first valid placement: one extra second of search captures most of the attainable quality, and a few tens of seconds are a conservative choice for the harder instances.

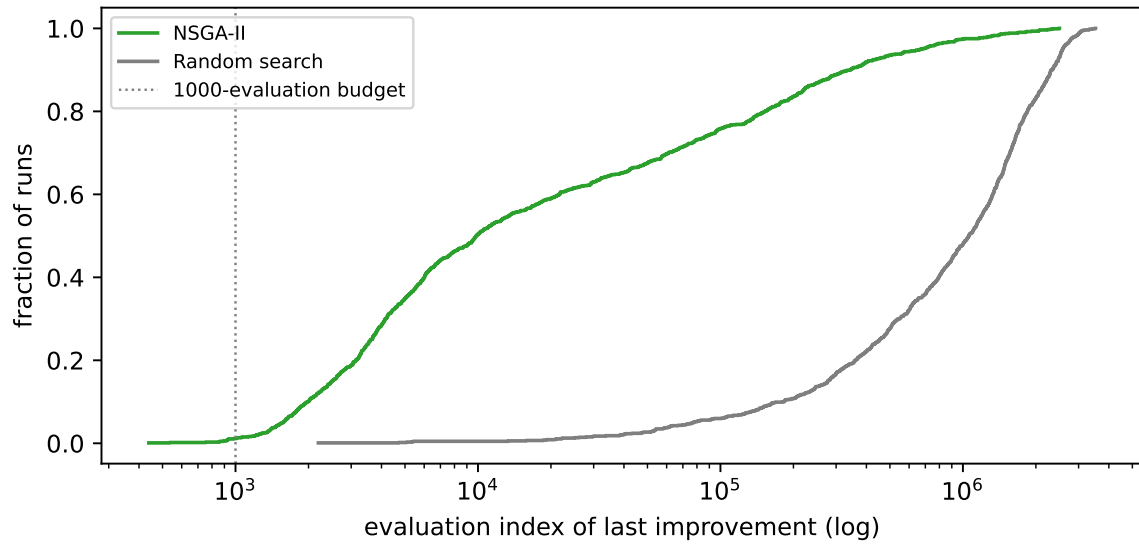


Figure 5.12: When does the last improvement happen? Distribution of the time of the last best-so-far improvement per engine. Improvements concentrating early are the empirical argument against pure time-based stopping.

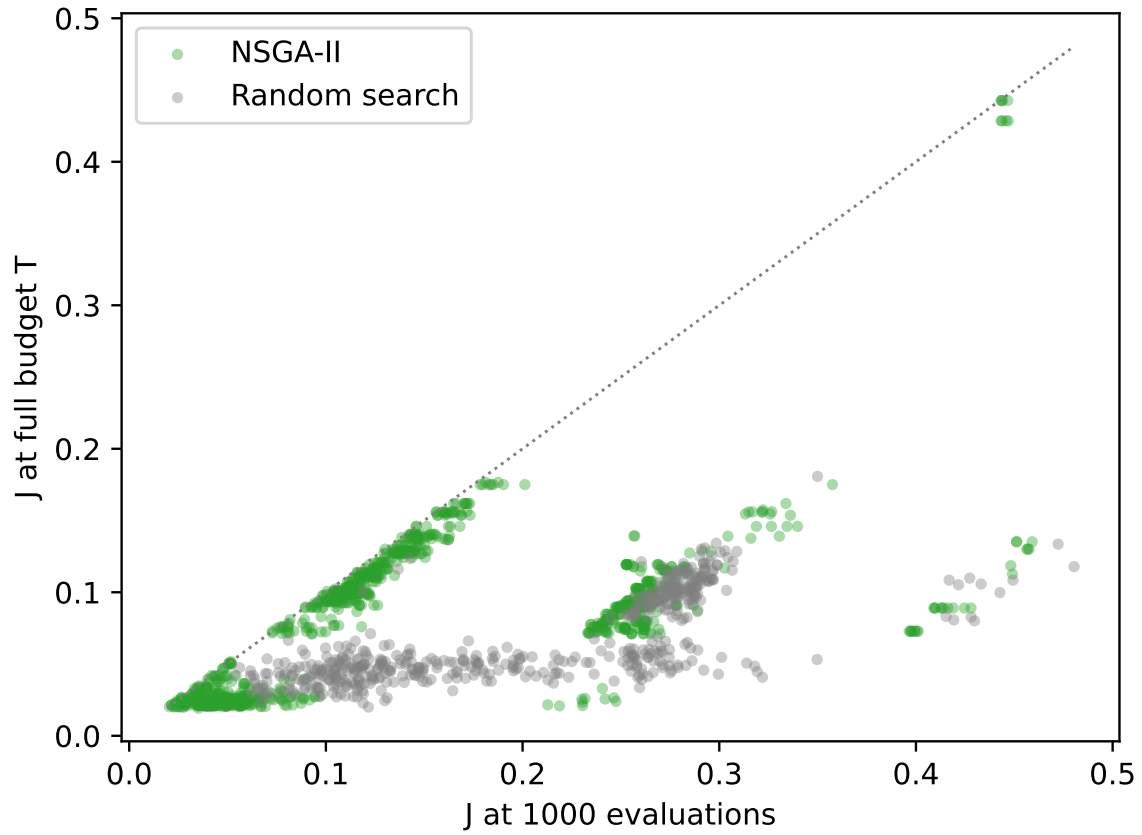


Figure 5.13: What the classic 1000-evaluation budget would have missed: objective at 1000 evaluations vs. at the full budget T , per engine (offline cutoff study from the traces).

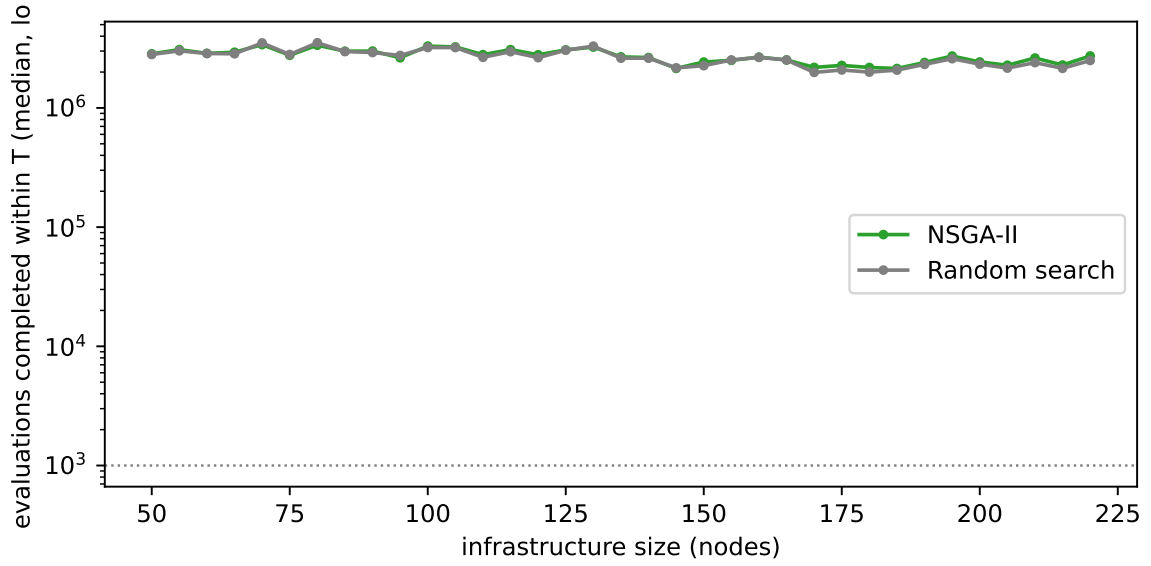


Figure 5.14: Budget utilization: evaluations completed within T per engine and infrastructure size (about 2.5 million for both stochastic engines).

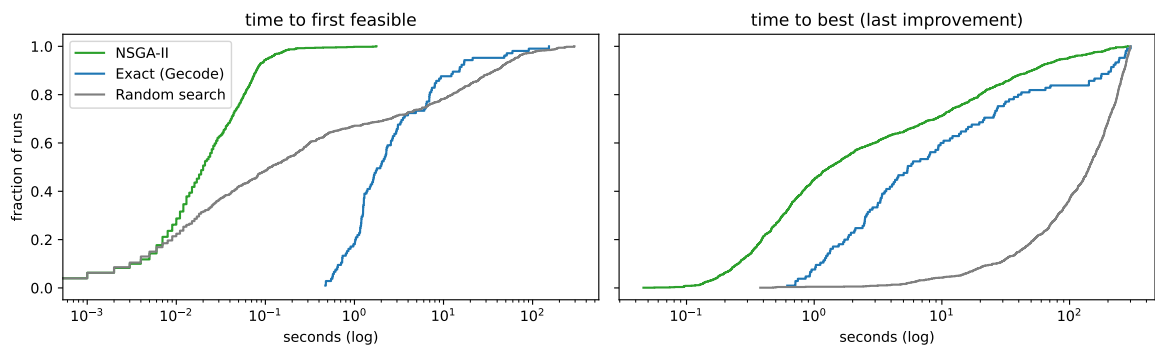


Figure 5.15: ECDFs of the time to the first feasible solution and the time to the best solution, per engine.

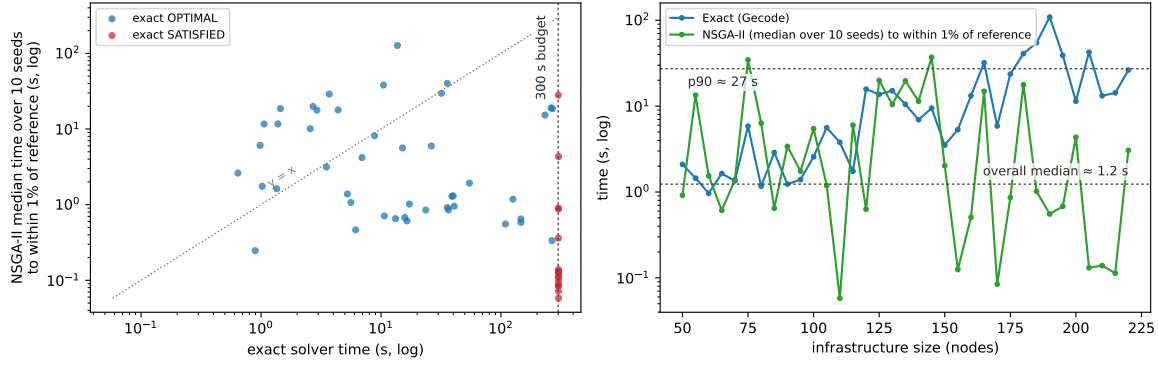


Figure 5.16: NSGA-II time to reach within 1% of the reference in most of its ten seeds: comparison with the exact engine’s own solving time, split by its completion status (left); median by infrastructure size (right).

Answers to **RQ₃**: How much planning time does the optimization require, and when is it worth continuing the search?

- (1) Stopping at the first valid placement forgoes most of the attainable quality, while a single second of further search recovers the bulk of it.
- (2) Budgets of seconds suffice for near-reference placements; exhausting the budget is only justified when an optimality certificate is needed.

5.5 RQ₄: Suitability for runtime rebinding

The quality–deadline trade-off of RQ₃ becomes operational when a replacement binding must be produced after a change in workload, pricing, or infrastructure conditions. NSGA-II produced its first feasible placement in less than 0.1 seconds in the median, providing an immediate fallback while the search continues. In 62 of the 105 instances, a majority of its ten seeds reached a solution within 1% of the exact reference, requiring a median of 1.2 seconds and a 90th percentile of 27 seconds (0.4 and 16 seconds under a 5% tolerance). Moreover, NSGA-II times did not systematically increase with infrastructure size (Fig. 5.16).

The comparison is also relevant for the 16 instances in which the exact engine exhausted its budget without a certificate: NSGA-II matched or improved the quality of the exact engine’s 300-second incumbent in every one of them, requiring a median of 0.13 seconds to reach that value — approximately 2400 times less solving time. The convergence examples of Fig. 5.17 and Fig. 5.18 illustrate this behavior on three representative instances spanning the size range: NSGA-II descends within the first seconds to near-reference quality, while the exact engine’s incumbent staircase catches up only when the proof machinery closes the instance.

These measurements concern only the computation of a new binding. They do not include the time required to detect a change, decide whether rebinding is necessary, migrate functions, or stabilize the new deployment. Within that scope, the results show that NSGA-II can provide an immediate feasible fallback and, for a substantial part of the benchmark, a near-reference placement within a planning budget of seconds.

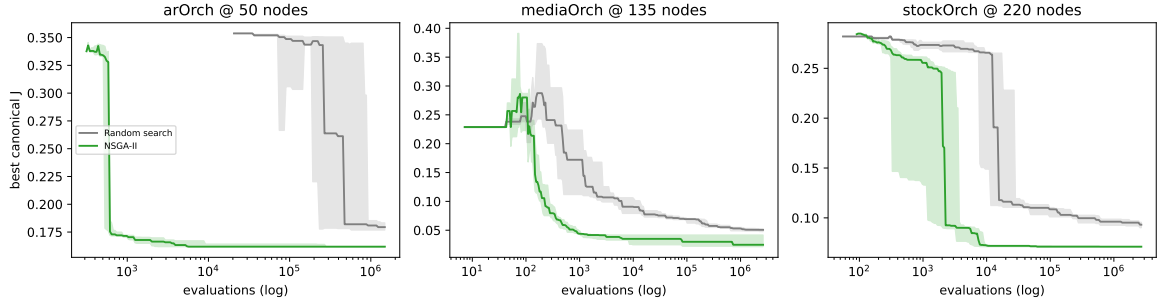


Figure 5.17: Best-so-far convergence vs. evaluations on three representative instances (median and IQR band over the stochastic seeds).

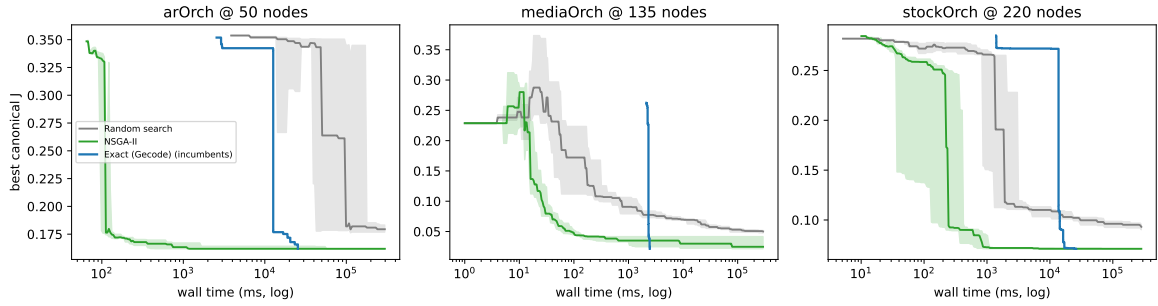


Figure 5.18: Best-so-far convergence vs. wall-clock time on the same instances, overlaying the exact engine's incumbent staircase.

Answers to **RQ₄**: Can the evaluated engines provide solutions quickly enough to support runtime rebinding?

- (1) NSGA-II provides a feasible placement almost immediately and often approaches the reference quality within short execution budgets.
- (2) Its runtime behavior supports rebinding in the planning stage, although the complete adaptation time also depends on monitoring and migration.

Chapter 6

Limitations and Threats to Validity

Construct validity. Our weighted-sum objective only reaches the convex region of the Pareto front and reports a single operating point. Security is a scalar aggregate of the countermeasures and trust relations inherited from [1], so richer information-flow properties may be lost. Candidate costs for cloud nodes come from iPricings and expected-consumption profiles rather than measured billing; our conclusions therefore concern relative comparisons under a common estimation model, robust to a shared bias.

Internal validity. We control heuristic stochasticity with ten seeds per instance and paired, scale-independent statistics. All engines share the same 300s budget, the normalization bounds stored in each BIM' instance, and an independent reference evaluator that rejects any value below a proven optimum; the oracle audit confirmed on all 2 205 runs that each engine optimized the canonical objective (zero mismatches). NSGA-II uses a single scalarized fitness with fixed (untuned) population and operators, so its reported quality is a lower bound. The stochastic lanes ran concurrently on a single host, which may introduce interference; budgets were enforced identically across lanes, solver time was isolated from gateway overhead, and the exact lane ran sequentially.

External validity. The benchmark reuses three orchestrations over 35 synthetic 50–220-node infrastructures from another study [1] and one corpus seed; other topologies, orchestration shapes, and real workloads may behave differently. We cover only on-demand billing for three providers (AWS Lambda, Azure Functions, Google Cloud Run functions), and the Pricing2Yaml multidimensional variables extension is experimental and not yet validated beyond them. The encoding, however, is engine- and domain-agnostic, and the full pipeline (generator, corpus, runner, and analyses) is available in the accompanying repository to ease replication on other datasets and engines (Appendix A).

Conclusion validity. With 10 repetitions per stochastic engine and instance, statistical power is limited. We therefore rely on non-parametric per-instance tests (Mann–Whitney U, Friedman) and co-report Vargha–Delaney $\hat{A}_{12}$ effect sizes, whose extreme values (median 1.0) make the conclusions robust to the sample size. Quality aggregations use feasible runs only; since feasibility rates are also reported (98.9–100%), the potential survivor bias is bounded and visible. Gains are conditioned on the chosen objective and weight vector, and runtime measurements cover only the computation of a new binding, so rebinding-suitability claims apply to the planning stage only.

Appendix A

Reproducibility Guide

This appendix summarizes the laboratory package: the repository layout, the commands that regenerate every artifact, and the datasets produced by the campaign. The experimentation is deterministic end to end: the generator seed (12345) plus the dataset seed (146588263) fully determine the corpus, solver seeds are explicit (1–10), run ids are deterministic (`instance|engine#seed`), and every reported objective is recomputed by the gateway reference evaluator.

A.1 Repository layout

```
experimentation/icsoc/
|-- original_dataset/      # SecFaaS2Fog input (applications + infrastructures)
|-- bimstar/              # BIM' generator package (+ unit tests)
|-- campaign.py            # resumable campaign runner (CLI + importable)
|-- analysis.py           # evaluation helpers (references, profiles, stats)
|-- notebooks/
|   |-- 01_dataset_preprocessing.ipynb # dataset -> corpus, audits
|   |-- 02_campaign_execution.ipynb    # stack, pilot, launch, monitoring
|   |-- 03_results_evaluation.ipynb    # full evaluation + figures
|-- report/               # this technical report (LaTeX)
|-- out/
|   |-- bimstar-priced/    # generated BIM' corpus + provenance reports
|   |-- results/          # runs.csv, traces.csv, corpus_summary.csv, figures/

pricings/                # FaaS iPricings (Pricing2Yaml + variables files)
schemas/general/         # general BIM schema + BIM' variant (bimstar.schema.json)
openbinding-gateway/     # gateway (validation, routing, reference evaluator)
engines/                 # minizinc-csp, random-search, evolutionary-heuristics,
                        # many-heuristic
```

A.2 Running the pipeline

Listing A.1: End-to-end execution (from the repository root).

```
# 0) Environment (once)
uv pip install -e ./openbinding-gateway pandas matplotlib scipy httpx \
  --python .venv/bin/python

# 1) Stack: gateway + engines
docker compose --profile dev up -d --build gateway-dev engine-minizinc \
  engine-random-search engine-evolutionary-heuristics engine-many-heuristic
```



```

# 2) Corpus (deterministic; also runnable from notebook 01)
.venv/bin/python -m experimentation.icsoc.bimstar.cli generate \
  --dataset experimentation/icsoc/original_dataset --pricing-dir pricings \
  --config experimentation/icsoc/bimstar/configs/default.yml \
  --seed 12345 --out experimentation/icsoc/out/bimstar-priced \
  --dataset-seeds 146588263

# 3) Campaign -- three parallel engine lanes (resumable)
.venv/bin/python experimentation/icsoc/campaign.py --engines minizinc-csp
.venv/bin/python experimentation/icsoc/campaign.py --engines random-search
.venv/bin/python experimentation/icsoc/campaign.py --engines evolutionary-heuristics

# 4) Monitor progress
.venv/bin/python experimentation/icsoc/campaign.py --status

# 5) Evaluate (produces every figure of Chapter 5)
jupyter lab experimentation/icsoc/notebooks/03_results_evaluation.ipynb

# Tests
pytest experimentation/icsoc/bimstar/tests openbinding-gateway/tests

```

The campaign can be sharded for a cluster by adding `-applications arOrch` (etc.) per shard and merging the CSVs afterwards; a shorter horizon can be set with `-time-budget-ms` (the ≥ 1000 -evaluation floor is preserved). Interrupted lanes resume where they stopped; deleting a row from `runs.csv` re-runs that combination.

A.3 Generated artifacts

- `out/bimstar-priced/instances/` — the 105 schema-valid BIM' instances (`*.bimstar.json`), organized by application and dataset seed.
- `out/bimstar-priced/reports/` — full provenance of the generation: `pricing_trace.csv` (per-candidate pricing derivation), `latency_trace.csv` (per-pair latency provenance), `security_trace.csv` (per-task threshold derivation), `budget_trace.csv` (witness costs, budgets, and normalization bounds), `candidate_generation_summary.csv` (per-task filter counts), and `validation_report.json`.
- `out/results/runs.csv` — one row per run (2 205 rows): status, feasibility, canonical and engine-internal objective values, `oracle_match` audit, per-feature values, solver time, evaluations, and completion status.
- `out/results/traces.csv` — one row per best-so-far improvement event ($\sim 47\,600$ rows): `eval_index` and `elapsed_ms`, enabling every offline cutoff analysis of Chapter 5.
- `out/results/corpus_summary.csv` — derived per-instance attributes (Table 4.1).
- `out/results/pilot_ga.csv` — the GA configuration pilot (Table 4.3).
- `out/results/figures/` — every figure of this report (`preprocessing_*` and `eval_*`, PDF and PNG).

Bibliography

- [1] Bocci, A., Forti, S., Ferrari, G.L., Brogi, A.: Declarative Secure Placement of FaaS Orchestrations in the Cloud-Edge Continuum. *Electronics* **12**(6), 1332 (2023)
- [2] Cavero, F.J., García-Fernández, A., Parejo, J.A., Resinas, M., Ruiz-Cortés, A.: Pricing Intelligence: Rethinking IS Engineering in Volatile SaaS Environments. In: *Advanced Information Systems Engineering. Lecture Notes in Computer Science*, vol. 16558, pp. 1–18. Springer Nature Switzerland (2026)
- [3] Cavero, F.J., Parejo, J.A., Ruiz-Cortés, A.: OpenBinding: An Extensible, Plug-and-Play API Gateway for Cross-Engine QACO Solving (2026). <https://doi.org/10.1109/ICWS72778.2026.00110>
- [4] Cavero, F.J., Parejo, J.A., Alonso, J.C., Ruiz-Cortés, A.: A-MINT: An LLM Pipeline for Automated Modeling of iPricings from SaaS Pricing Pages. In: *Service-Oriented Computing. ICSOC 2025. LNCS*, vol. 16321, pp. 83–91. Springer (2026)
- [5] Deb, K.: An efficient constraint handling method for genetic algorithms. *Computer Methods in Applied Mechanics and Engineering* **186**(2–4), 311–338 (2000)
- [6] Deb, K., Pratap, A., Agarwal, S., Meyarivan, T.: A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation* **6**(2), 182–197 (2002)
- [7] Dolan, E.D., Moré, J.J.: Benchmarking optimization software with performance profiles. *Mathematical Programming* **91**(2), 201–213 (2002)
- [8] Friedman, M.: The use of ranks to avoid the assumption of normality implicit in the analysis of variance. *Journal of the American Statistical Association* **32**(200), 675–701 (1937)
- [9] García-Fernández, A., Parejo, J.A., Cavero, F.J., Ruiz-Cortés, A.: Trends in Industry Support for Pricing-Driven DevOps in SaaS. *IEEE Transactions on Services Computing* **19**(1), 726–737 (2026)
- [10] Matricardi, A., Bocci, A., Forti, S., Brogi, A.: Simulating FaaS Orchestrations in the Cloud-Edge Continuum. In: *Proceedings of the 3rd Workshop on Flexible Resource and Application Management on the Edge*. pp. 19–26. FRAME '23, Association for Computing Machinery (2023)
- [11] Nethercote, N., Stuckey, P.J., Becket, R., Brand, S., Duck, G.J., Tack, G.: MiniZinc: Towards a standard CP modelling language. In: *Principles and Practice of Constraint Programming (CP 2007). Lecture Notes in Computer Science*, vol. 4741, pp. 529–543. Springer (2007)

- [12] Parejo, J.A., Segura, S., Fernandez, P., Ruiz-Cortés, A.: QoS-aware web services composition using GRASP with Path Relinking. *Expert Syst. Appl.* **41**(9), 4211–4223 (2014)
- [13] Vargha, A., Delaney, H.D.: A critique and improvement of the CL common language effect size statistics of McGraw and Wong. *Journal of Educational and Behavioral Statistics* **25**(2), 101–132 (2000)